Sharing Without Splitting: Concurrent Conflict Weights for Parallel Crossword Filling

Luke K. Schreiber
Undergraduate Student, Department of Computer Science
Princeton University
Princeton, NJ, USA
ls1881@princeton.edu

***Abstract*—Parallel constraint satisfaction problem (CSP) solvers typically partition the search space across workers or run an undivided portfolio of independent restarts; the closest prior work to combine the two treats sharing as a one-time input to partitioning, not an ongoing signal. This paper describes a restart portfolio whose workers never partition the problem, but continuously and concurrently update a shared pool of per-constraint conflict weights, with no manager and no synchronization phase, implemented in xfill, a word-level CSP solver for crossword filling. Restart-portfolio search occupies a genuinely different point in the design space than partition-based search, represented here by orca-solver, an independent alternative. Oversubscribing thread count improves xfill roughly 65 times on one hard grid, while orca-solver improves only marginally and cannot solve it below 14 threads. On 12 valid 15x15 grids, xfill solves every one, and is faster on 9 of the 11 where orca-solver also succeeds; across all 11, the geometric mean speedup is 107 times. Partitioning, conversely, gives non-overlapping coverage useful for proving unsatisfiability. Isolating the mechanism itself shows a consistent win on moderate-difficulty grids but higher variance rather than a reliable one on the hardest instances, reported alongside the positive result. A further negative result: five plausible branching heuristics produced identical output because a domain-size precondition meant none ever executed, caught only by direct instrumentation, not benchmark comparison.**

***Keywords*—constraint satisfaction, parallel search, restart portfolios, dom/wdeg heuristic, crossword generation**

I. Introduction

Crossword filling (assigning a dictionary word to every slot in a grid so every crossing letter agrees) is a word-level constraint satisfaction problem (CSP) and a convenient search-research testbed: instances are human-legible, difficulty is controllable directly through grid geometry, and independently developed solvers already exist to compare against. Ginsberg's Dr.Fill [1] established crossword filling as a serious weighted-CSP application over a decade ago. In the practical workflow of a crossword constructor, automated filling tools frequently bottleneck on wide-open grid sections, significantly slowing the iterative design process.

This paper proposes xfill, a crossword filling algorithm that answers one question about parallel constraint search: when multiple workers run concurrently, is there value in sharing *soft* heuristic statistics (information that biases search order without asserting anything logically) continuously, without ever partitioning the problem those workers are solving? Section II shows this combination does not exist in prior work; Section III implements it; Section V measures its effect, both in isolation and against the broader architecture.

Contributions: (1) a concurrent, decentralized conflict-weight-sharing mechanism for restart-portfolio CSP search, positioned precisely against the closest prior work; (2) a reproducible four-solver testbench that separates two claims: restart-portfolio search as an architectural class, and this specific mechanism within it; and (3) a methodological negative result about confirming a heuristic executes before trusting its benchmark comparison.

II. Background and Related Work

A CSP assigns values to variables from finite domains subject to constraints; systematic backtracking assigns one variable at a time, then propagates, removing now-impossible values from other variables' domains, and backtracks if some variable's domain empties entirely (a *wipeout*). The dom/wdeg (domain-over-weighted-degree) heuristic [4] selects the variable with smallest domain-size-to-weighted-degree ratio, where a constraint's weight increases on every wipeout it causes. Randomized backtracking exhibits heavy-tailed runtime [2], [3]: restarting periodically eliminates most of the tail and is standard in Boolean satisfiability (SAT) and CSP solvers alike.

Two dominant parallel strategies exist. *Portfolio* approaches run independent searches over the whole problem concurrently; SAT portfolios such as ManySAT [6] additionally share *learned clauses* (facts inferred from a worker's own failed attempts) between workers without partitioning. *Partitioning* approaches divide the search space so each worker explores a disjoint region, giving non-overlapping coverage useful for proving unsatisfiability (that no solution exists, UNSAT).

Three prior CSP results sit near, but not on, the point in this design space described here. Grimes and Wallace [5] accumulate dom/wdeg-style weights from short restarted probes of a *single sequential* solver, never concurrent. Ehlers and Stuckey [7] parallelize a lazy-clause-generation solver, concurrently sharing learnt clauses and objective bounds between workers – *hard* facts, unlike this paper's soft weights – and requiring clause-learning machinery most CSP solvers lack. Yun and Epstein's SPREAD [8] is the closest: a manager

979-8-3195-2794-3/26/$31.00 ©2026 IEEE

races workers under a plain dom/wdeg solver, then *averages* their reported weights once each exhausts a budget, using the average to *partition* the problem for a subsequent phase. The authors state this directly: "the primary purpose of SPREAD's portfolio phase is to glean information to support search space splitting, not to solve P" [8]. A comprehensive 2018 survey [9] documents concurrent, non-partitioning multi-agent search sharing hard facts (learned clauses, as in ManySAT) or concrete candidate-value hints, but no work sharing continuously updated soft heuristic weights among such workers.

The mechanism in Section III occupies the gap these leave: concurrent, unlike Grimes and Wallace; soft, unlike Ehlers and Stuckey; and never used to partition, unlike Yun and Epstein.

III. System Architecture

xfill parses a grid into slots (open runs ≥ 2 cells) and crossings (cell-level intersections). Its sequential core performs dom/wdeg-style branching by selecting the slot minimizing domain size over accumulated *crossing weight*, then propagates letter constraints via an AC-3-style (Arc Consistency Algorithm #3) queue (a standard constraint-propagation algorithm); a domain wipeout increases the responsible crossing's weight.

`SolveParallel` runs N independent restart workers, each with its own seed and private, decaying crossing-weight table, plus one worker dedicated to a single uninterrupted exhaustive search (`unlimited_budget`) that guarantees eventual completion independent of restart variance. Since a restart worker's budget only ever grows and never discards earlier work, its search is complete in the limit too; any worker's mathematically valid negative result therefore ends the others' searches.

Concurrent conflict-weight sharing. Alongside each worker's private table, `SolveParallel` maintains one array of plain atomic counters (thread-safe, uninterruptible updates), one per crossing, shared by every restart worker except the dedicated exhaustive one. Every worker increments a crossing's counter on wipeout and reads every counter when scoring candidates. The counters are plain, order-independent atomic increments rather than a decaying scheme: a decaying normalizer is safe only under a single writer, since it depends on the exact sequence of prior updates; concurrent writers need a rule that ignores order entirely. The dedicated exhaustive worker is excluded because its entire value is one undisturbed trajectory. Wiring it into the shared signal regressed one grid in the corpus, and excluding the worker again did not recover that grid either: the shared signal itself, not solely the dedicated worker's presence in it, had redirected ordinary restart workers' own search order. Two correctness properties were required: single-threaded calls stay byte-identical to the unshared baseline, since the mechanism only activates for thread count > 1; and ThreadSanitizer reports no unsafe concurrent access under real multi-threaded solves. In earlier development-time testing across a broad corpus of moderate-difficulty grids (`docs/design.md`), this mechanism cut solve time broadly, revisited against a harder, targeted ablation in Section V-B.

IV. A Negative Result: Branching on the Wrong Thing

Motivated by a 7×7 grid known to be exceptionally hard for restart-based search, five letter-level branching variants were implemented (three minimum-remaining-value scoring rules, a faithful reimplementation of a partition-based solver's own candidate scoring, and a reversed scan order), reasoning that a smaller per-node branching factor should help on wide-open grids.

All five produced numerically identical node and backtrack counts, too strong a signal to be coincidence, prompting further investigation. A plain counter in the branch-selection code, tracking how often the large-domain branch these heuristics govern actually executed, showed it firing *zero* times across more than 17,000 real search nodes: propagation from the grid's pre-filled letters narrowed every domain below the branch's activation threshold within the first few assignments. All five were unreachable code the entire time they were compared, which is why their benchmark scores could not have distinguished them. The reusable point: an A/B comparison of branching heuristics is only informative once the branch under test is confirmed, by direct instrumentation, to execute. This negative result directly informs better design practices for future parallel CSP solvers, ensuring benchmarking efforts are spent on genuinely active code paths rather than phantom optimizations.

V. Experiments

A. Setup

The experiments below test two distinct claims kept separate rather than blended: Section V-B isolates the mechanism itself first, holding architecture fixed, since that is this paper's actual contribution. Sections V-C–V-F then test the broader architectural claim from the Introduction directly, using orca-solver as a concrete, independently developed stand-in for partition-based search, and an unmodified naive backtracker to show the comparison is not trivial.

Solvers. xfill (this work); orca-solver [12], a partition-based parallel solver treated as a black box; ingrid_core [13], a single-threaded reference solver; and, as a naive baseline with no restarts, no learned heuristics, and no parallelism, crossword-composer [14] (a static-constraint-order Rust backtracker). All four run through a single fixed harness (`benchmarks/urtc2026_testbench/`) in the paper's code repository [15], against a dictionary derived deterministically from a public word list [11], with a fixed seed and a 300-second cap, reproducible via `run_benchmark.py`. Every timeout was re-tested at 600s, then again at 1000s, via `rerun_timeouts.py` (Section V-E). xfill and orca-solver race independent workers, so wall time varies run to run; each runs 5 times per grid, median reported, every trial logged. ingrid_core and crossword-composer are single-threaded and deterministic, so each runs once.

Grid set. A size-graded curated set (5×5 through 21×21) plus a fixed-seed random sample of 15×15 grids scraped from a public corpus of valid crossword layouts, crossword-grids.com [10].

B. Isolating the Mechanism: A Shared-Weight Ablation

This section tests xfill against itself: same solver, same grid, same dictionary, same 14 threads, with one environment variable flipped (`XFILL_DISABLE_SHARED_WEIGHTS` on and off) and several trials each. Grids come from two independent sources, to rule out an artifact of one curated list: a fixed "known-hard" grid list reused from this project's own earlier development-time benchmarking (not cherry-picked for this paper; a different subset of the same master list anchors the thread-count scaling result in Section V-C), and three grids from the paper's own main scraped-15×15 corpus.

Fig. 1 does *not* show a uniform win on either grid set. On `grid_053`, the unshared baseline is faster on the median (0.72s vs. 2.77s) and far tighter (0.66–0.76s vs. 0.07–3.71s). On `grid_058`, medians are nearly identical (0.43s vs. 0.44s) but shared weights show wider spread; `grid_479` shows the same pattern plus a mild increase in the median (5.72s to 7.76s). On `grid_120`, shared weights reach a lower best case but also produce the only outright timeout among those eighteen trials. `grid_395` and `grid_423` sharpen both directions: a clean $\sim 7\times$ win on the former (36.9s vs. 271.4s median), and actively worse on the latter, with all 5 trials hitting the 300s cap, where disabling shared weights solves it reliably at $\sim$180s every time. `grid_423` reappears in Section V-F, from a separate wide-spread sample, as the grid where xfill uniquely completes what orca-solver cannot – a completion advantage from exactly this unreliable, default-on setup. Against Section III's broader-corpus finding of a consistent win, the result is regime-dependent: shared weights help on typical grids by redistributing which crossings each worker deprioritizes, but at the extreme difficulty end trade tight, well-behaved variance for a wider one that is not reliably better, and sometimes worse. There is no evidence for a uniform win, so none is claimed; the same standard Section IV applies to the heuristics it rules out.

C. Restart Portfolios Exploit Added Parallelism

Restart-portfolio search should benefit from more independent workers even past the physical core count, because heavy-tailed runtime means each additional worker is another draw that might get lucky early. This was tested directly on three grids known to be hard for restart-based search, oversubscribing threads well past this machine's 14 physical cores (Fig. 2).

D. Is This a Property of the Architecture, or One Grid?

This further test splits three ways, showing this is a property of the architecture rather than a lucky pick of grid. On `grid_120` (Fig. 2), xfill's wall time falls from 27.8s at 1 thread to 0.43s at 42 (a $65\times$ improvement), while orca-solver never solves it below 14 threads and only creeps from 27.3s to 17.9s from 14 to 42. On `grid_303` (Fig. 2), xfill solves it at 8, 14, and 42 threads, with wall time actually increasing across those three (22.9s to 38.2s), while orca-solver fails to solve it at *any* of the 7 thread counts tested, so the win here is completion, not speed. On `grid_115` (timed out at every thread count for both solvers, so omitted from the figure), neither solver succeeds at all, and thus this one favors neither architecture. Where the pattern holds, it is because absorbing extra parallelism without restructuring the search is what a non-partitioning restart portfolio is built to do; a partition-based design does not exhibit that capacity to nearly the same degree.

TABLE I
SOLVE OUTCOMES ACROSS ALL 20 GRIDS, 1000S CAP (DECIDED = SOLVED + UNSAT)

Solver	Solved	UNSAT	Timeout	Decided
xfill	18	1	1	19
orca-solver	17	1	2	18
ingrid_core	16	1	3	17
crossword-composer	8	1	11	9

E. Naive Backtracking Is Not a Baseline Worth Beating

Zooming out to the full sweep across all four solvers and every curated grid size (Fig. 3), the same advantage holds broadly, testing directly whether crossword-composer's losses are a genuine baseline failure or just an under-provisioned time budget. Table I summarizes the outcome: across all 20 grids, xfill decided 19, orca-solver 18, ingrid_core 17, and crossword-composer only 9 – a real, unmodified, independently authored solver, not a strawman built to lose.

Escalating the cap from 300s to 600s changed exactly one outcome: ingrid_core went on to solve `grid_479` in 325s. A further escalation to 1000s (the cap these figures use) changed none: every solver's remaining timeouts fall within crossword-composer's same 11 grids, including `sample_13x13` (the one grid no solver resolves at any cap), and all held: evidence these are hard, not under-budgeted. The point for this paper's thesis is narrower than "xfill wins": restarts, adaptive heuristics, and parallelism are not optional refinements at these sizes, but the premise this paper's mechanism depends on.

F. Aggregate Speed Advantage Where Both Solve

Fig. 4 quantifies the restart-portfolio/partition comparison on the scraped 15×15 corpus alone [10], using realistic-scale grids, rather than the curated size-graded set, which mixes in small grids that inflate the ratio. Of the 11 scraped grids (of 12) both xfill and orca-solver solve, xfill is faster on 9 and slower on 2 (`grid_395` and `grid_479`, by $5.4\times$ and $1.6\times$); across all 11, the median speedup is $494\times$ and the geometric mean $107\times$. This is consistent with `grid_115` in Section V-D: not every grid favors either architecture. `grid_423`, the twelfth scraped grid, sharpens this in the opposite direction: xfill solves it in 270s where orca-solver cannot, even at this paper's full 1000s cap, thus representing a completion advantage, not just a speed one (revisited in Section V-B). Across the full 20-grid sample, xfill decides 19, orca-solver 18, both agree on the sample's one correct UNSAT proof, and xfill's one failure (`sample_13x13`) is a subset of orca-solver's two, `grid_423` being the other. There is a narrower gap on success alone than

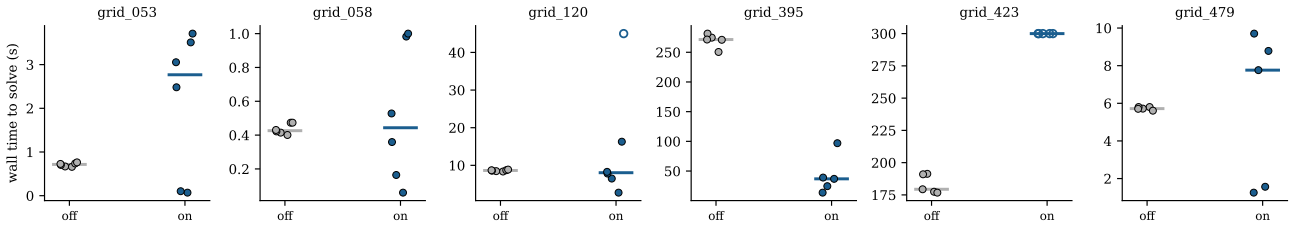


Fig. 1. Shared-conflict-weight ablation, off vs. on, 14 threads: a known-hard grid list also used in Fig. 2 (left three, 6 trials/config, 45s cap) and three grids from the paper's own scraped-15 × 15 corpus (right three, 5 trials/config, 300s cap; the other 9 of 12 solve in under a second and are omitted). Bars mark the median; hollow points hit the cap. Shared weights are not uniformly better: the same win/wash/loss pattern appears independently on both grid sets.

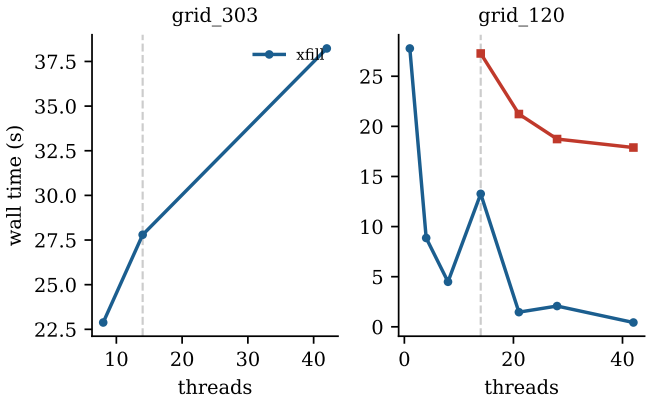


Fig. 2. Wall time vs. thread count, xfill vs. orca-solver, on two grids known to be hard for restart-based search (45s cap; dashed line = 14 physical cores; orca-solver missing from a panel means all 7 of its thread counts timed out; a third grid, grid_115, times out for both solvers at every thread count tested and is omitted, see text). grid_120 is the clearest case: xfill improves roughly 65× from 1 to 42 threads while orca-solver stays flat and cannot solve at all below 14.

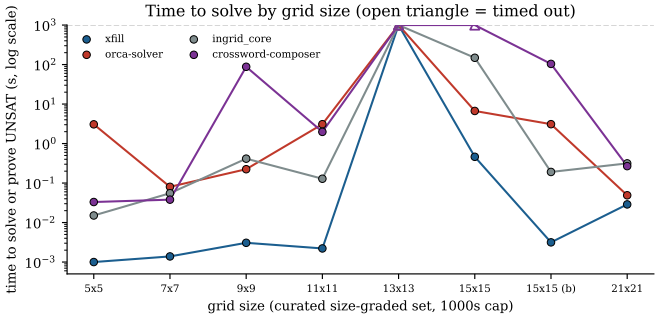


Fig. 3. Time to solve or prove UNSAT by grid size, curated set, 1000s cap (log scale; open triangle = timed out; a second, denser 15 × 15 grid is marked (b)). A solved/not-solved view of this same data would show xfill, orca-solver, and ingrid_core essentially tied, since all three succeed on the same grids here: the actual difference is that xfill's line sits one to nearly five orders of magnitude below the others at every size except 13 × 13 (every solver times out together) and 21 × 21 (an easy UNSAT proof every solver resolves in well under a second regardless).

on speed, which is why Sections V-D and V-B both turn to a harder, pre-existing grid list for a more demanding test.

VI. Discussion

Both results share one underlying cause: restart portfolios benefit from heavy-tailed runtimes, so more independent draws raise the chance of an early lucky one. This is why oversubscribing thread count helped in Section V-C and Fig. 2, and why `grid_423` (Fig. 4) goes further, succeeding where a fixed partition cannot. Partitioned search instead gives every worker a disjoint slice of the space to rule out, which is exactly what makes it valuable for proving unsatisfiability (Section II). The same heavy-tailedness explains Fig. 1's mixed result: shared conflict weights change which draw from the tail each worker takes, which can land on a worse draw as easily as a better one, netting out as an improvement only once averaged over a broader, typical corpus.

Five branching heuristics (Section IV) looked interchangeable only because none ever executed on the grid that motivated them, a gap direct instrumentation closed but benchmark-score comparison alone would have missed. The same standard applies to the ablation in Section V-B: reporting the broad-corpus win alone, without also checking the harder grids directly, would have missed the regime-dependence that turned out to matter most. In both cases, the paper's numbers were trustworthy only once verified against what the code actually did, not just its aggregate score.

VII. Limitations

All timing results are from one 14-core machine and one dictionary derived from a single external word list; the thread-scaling numbers should not be read as hardware- or dictionary-independent claims. The architectural comparison also rests on one partition-based representative, orca-solver, chosen for being fast, open-source, and actively maintained rather than through a systematic survey of partition-based solvers; results may not generalize to every such implementation. ingrid_core is likewise active; the version or commit used for each solver is recorded in the testbench README.

VIII. Conclusion

This paper's contribution is the mechanism in Section III: concurrent conflict-weight sharing across a restart portfolio that never partitions, occupying a gap that three adjacent lines of prior work leave open. The architectural comparison in Section V supports that gap's importance rather than being

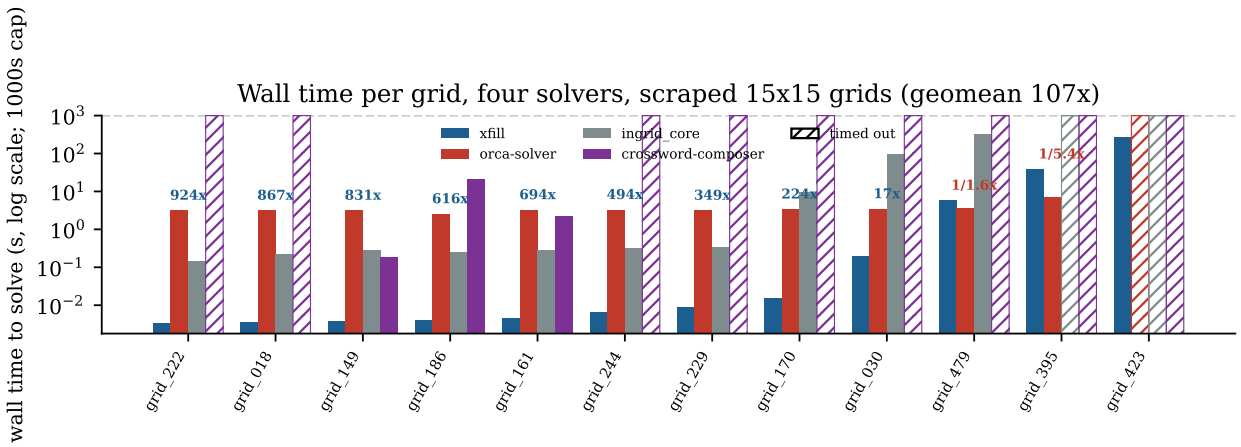


Fig. 4. Wall time to solve (log scale, 1000s cap), four solvers, per scraped 15×15 grid, sorted by xfill's time. Hatched bars mark a timeout; crossword-composer times out on most grids, the same pattern reported by count in Section V-E. Label above the xfill/orca-solver pair is orca-solver time / xfill time; two grids (`grid_395`, `grid_479`) favor orca-solver, and on `grid_423` orca-solver cannot solve it even at this cap while xfill does, in 270s.

the paper's subject: restart portfolios suit finding solutions quickly, partitioned search suits proving none exist, a pattern independent of any one solver. The mechanism itself, tested in isolation, is more modest than the architecture it sits inside: a real win on typical grids, a wash or worse on the hardest ones, reported as measured rather than as hoped. A negative result was also reported from the same codebase, in which five heuristics were rendered indistinguishable by a precondition that kept them from ever executing, illustrating a reusable caution for comparing branching heuristics by benchmark score alone. Although xfill is built for crossword filling, its mechanism is not: it needs only a dom/wdeg-style weight table and a restart loop, both common in CSP and lazy-clause-generation solvers, so the same approach could extend to other CSPs, the combination that Section II found no prior system using.

Future work could test that extension directly, in a non-crossword CSP domain such as graph coloring, Sudoku, or job-shop scheduling (assigning factory jobs to machines and time slots without conflicts), where a wdeg-style solver already exists and only the shared-weight array would need to be added. This could significantly accelerate resource-constrained scheduling tasks without requiring heavy synchronization overhead. It could also broaden the architectural comparison to other partition-based solvers, and re-run the ablation on a genuinely independent machine and dictionary, to separate what is specific to this setup from what generalizes.

Acknowledgment

The author thanks the Department of Computer Science at Princeton University for funding that made possible the presentation of this paper at IEEE MIT URTC. The author used Claude Code (Anthropic), an AI coding assistant, to help build this project. The author takes full responsibility for all code, experimental results, and conclusions in this paper.

References

[1] M. L. Ginsberg, "Dr.Fill: crosswords and an implemented solver for singly weighted CSPs," *Journal of Artificial Intelligence Research*, vol. 42, pp. 851–886, 2011.

[2] C. P. Gomes, B. Selman, and H. Kautz, "Boosting combinatorial search through randomization," in *Proc. AAAI*, 1998, pp. 431–437.

[3] C. P. Gomes, B. Selman, N. Crato, and H. Kautz, "Heavy-tailed phenomena in satisfiability and constraint satisfaction problems," *Journal of Automated Reasoning*, vol. 24, pp. 67–100, 2000.

[4] F. Boussemart, F. Hemery, C. Lecoutre, and L. Sais, "Boosting systematic search by weighting constraints," in *Proc. ECAI*, 2004, pp. 146–150.

[5] D. Grimes and R. J. Wallace, "Sampling strategies and variable selection in weighted degree heuristics," in *Proc. CP*, LNCS vol. 4741, 2007, pp. 831–838.

[6] Y. Hamadi, S. Jabbour, and L. Sais, "ManySAT: a parallel SAT solver," *J. Satisfiability, Boolean Modeling and Computation*, vol. 6, pp. 245–262, 2009.

[7] T. Ehlers and P. J. Stuckey, "Parallelizing constraint programming with learning," in *Proc. CPAIOR*, LNCS vol. 9676, Banff, AB, Canada, 2016, pp. 142–158.

[8] X. Yun and S. L. Epstein, "A hybrid paradigm for adaptive parallel search," in *Proc. CP*, LNCS vol. 7514, 2012, pp. 720–734.

[9] I. P. Gent, I. Miguel, P. Nightingale, C. McCreesh, P. Prosser, N. C. A. Moore, and C. Unsworth, "A review of literature on parallel constraint solving," *Theory and Practice of Logic Programming*, vol. 18, no. 5–6, pp. 725–758, 2018.

[10] Crosswordgrids.com. "15x15 crossword grids." Accessed: Jul. 29, 2026. [Online.] Available: https://crosswordgrids.com/15x15-crossword-grids

[11] B. Husic and E. Henestroza Anguiano. "Spread the word(list)." Spreadthewordlist.com. Accessed: Jul. 22, 2026. [Online.] Available: https://www.spreadthewordlist.com/

[12] J. Hawksley. *orca-solver*. (2026). Version 0.3.0. Accessed: Aug. 5, 2026. [Online]. Available: https://github.com/rainjacket/orca-solver

[13] R. Fitzgerald. *ingrid_core*. (2026). Version 1.3.1. Accessed: Aug. 5, 2026. [Online]. Available: https://github.com/rf-/ingrid_core

[14] P. Butler. *crossword-composer*. (2026). Commit 912c5ee. Accessed: Aug. 5, 2026. [Online]. Available: https://github.com/paulgb/crossword-composer

[15] L. K. Schreiber. *xfill: a word-level CSP solver for crossword filling*. (2026). Source code, benchmarks, and results. Accessed: Aug. 7, 2026. [Online]. Available: https://github.com/ls1881/crossword-filler